LLM APIs Deep Dive & Models

Boolean Master · Lezione 13 · Modulo 2 · 2 ore



Dove siamo: si apre il Modulo 2

Alla L12 abbiamo fatto la prima chiamata, in Python, in un progetto locale. Oggi due movimenti.

MODULO 2 · LLM & DATA

↔ ZOOM-OUT · IL PANORAMA

Chi fa i modelli, il mercato, i prezzi

Chi addestra i modelli, closed vs open-weights, le quote di mercato, quanto costa, quale modello per cosa.

⤢ ZOOM-IN · DENTRO LA RESPONSE

Il deep dive che avevamo rimandato

La struttura completa della risposta e dei parametri: alla L12 l'avevamo esplicitamente rinviato a oggi.

Lezione frontale: niente live coding da zero. Concetti, loghi, grafici e tabelle, e un walkthrough della documentazione.

Programma della serata

Otto tappe: prima il panorama, poi il deep dive tecnico.

01
Chi addestra i modelli

02
Il mercato e le aree

03
Pay-per-token

04
Quale modello per cosa

05
I prezzi

06
La documentazione

07
La chiave API

08 · IL CUORE
Dentro la response

1. Chi addestra i modelli

Noi facciamo inferenza, non training

Il training non lo facciamo noi

Lo avete già visto alla L10 e alla L12: il modello lo addestra qualcun altro.

TRAINING · I GROSSI PLAYER

Addestrare un modello

Servono dati enormi, mesi di compute su migliaia di GPU e capitali enormi. Lo fanno in pochi al mondo.

→

INFERENZA · NOI

Usare un modello già addestrato

Mandiamo un input via API e otteniamo una risposta. È quello che abbiamo fatto alla L12.

IN BREVE Il modello lo ha già addestrato qualcun altro: noi lo usiamo, e basta — questa è l'inferenza.

I player: closed-source vs open-weights

Due famiglie di aziende che addestrano i modelli di punta.

CLOSED-SOURCE · pesi chiusi, solo via API

- OpenAI
- Anthropic
- Google · Gemini
- xAI · Grok

OPEN-WEIGHTS · rilasciano i pesi

- Meta · Llama
- Mistral
- DeepSeek
- Alibaba · Qwen
- Moonshot · Kimi
- e altri (Z.ai/GLM…)

Oggi i modelli di punta su molte classifiche sono **closed-source** — ma gli **open-weights** li hanno ormai raggiunti, e su diversi task sono pienamente competitivi.

Allora perché sceglierne uno closed e pagare le API?

Cosa vuol dire "open weights"

Rilasciano i pesi del modello. Da lì, due strade.

SELF-HOST · la tua GPU

Scarichi i pesi e fai girare il modello sul tuo hardware. Niente pay-per-token, ma ti procuri e gestisci la GPU.

API · loro o un inference provider

Le stesse aziende offrono anche le API. E ci sono provider terzi che servono i modelli open via API: **Groq**, **Together**, **Fireworks**, **OpenRouter**.

Attenzione ai nomi: `Groq` (con la q) è un **inference provider** per modelli open — non è `Grok` (con la k) di xAI, e non addestra modelli.

IN BREVE Open weights = puoi scaricare i pesi e farli girare tu; ma quasi sempre conviene comunque l'API.

API (closed) o self-hosting (open)?

La domanda di prima, presa sul serio: le due strade a confronto.

API · closed, o anche open gestito da un provider

✓ Pro

- Niente infrastruttura: nessuna GPU da comprare o gestire
- Affidabilità e scaling a carico del provider
- Cambi modello con una riga: provi, confronti, aggiorni
- Paghi a consumo: **$0 quando non la usi**

✕ Contro

- I dati passano da terzi (data retention, compliance)
- Dipendi dal provider: prezzi, rate limit, disponibilità

SELF-HOSTING · open weights sulla tua GPU

✓ Pro

- I dati restano in casa: privacy, data residency
- Nessun pay-per-token; controllo totale e uso offline

✕ Contro

- GPU costose (acquisto o noleggio) + elettricità, raffreddamento, manutenzione
- Paghi **anche quando è ferma**; gestisci scaling e aggiornamenti
- Uptime e performance sono problema tuo

Cos’è una GPU e perché serve

Un modello sono miliardi di parametri: l'inferenza è tanta algebra lineare, tutta in parallelo.

PARALLELISMO

Migliaia di core

La GPU fa moltissime moltiplicazioni di matrici insieme: è esattamente ciò che serve a un modello.

VRAM

Decine di GB

Per tenere i pesi in memoria servono decine o centinaia di GB di memoria video ad alta banda.

COSTO

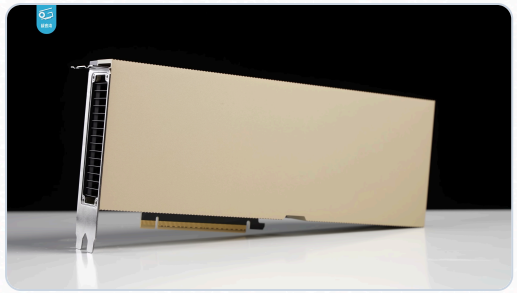
Migliaia di €

Hardware costoso da comprare e da gestire (alimentazione, raffreddamento, aggiornamenti).

Ma quanto costa **davvero** tenersi le GPU in casa? Mettiamo i numeri. →

Self-hosting: i numeri veri

Per far girare un buon modello open (~70B) a velocità decente serve hardware serio.



NVIDIA H100 · foto Geekerwan, CC BY 3.0

VRAM

~140 GB

un 70B in FP16 → servono **2-4× H100** (80 GB l'una)

PREZZO

~$30-40k

a GPU; o ~$2-4/h a noleggio → **~$2.200/mese** 24/7 (×2-4 GPU)

VELOCITÀ

~250-460

tok/s su 1× H100, ma **aggregati** su molti utenti: il singolo ne sente molti meno

Nel **~95% dei casi conviene l'API**: un Llama 70B gestito costa **$0.35-0.88 / 1M token** (e $0 da fermo). Self-host solo per **privacy/data-residency, volumi enormi, fine-tuning o uso offline**.

Fonti: CloudZero, IntuitionLabs, Artificial Analysis, Cerebrium · rilevazione 22/06/2026.

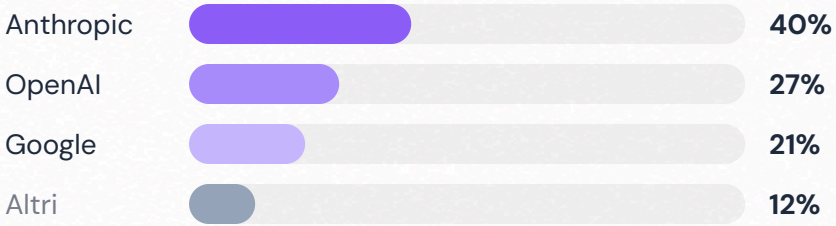
2. Il mercato e le aree di forza

Chi fa cosa nella sfera genAI

Le fette di mercato

Il mercato è concentrato su pochi nomi — ma "share" dipende da cosa misuri.

SPESA API ENTERPRISE · 2025



Sul **coding**: Anthropic ~54%, OpenAI ~21%.

UN ALTRO SEGNALE · VOLUME TOKEN DEV

Misurando il **volume di token** instradato dagli sviluppatori (OpenRouter), i modelli **open cinesi** (DeepSeek, Qwen, Kimi…) valgono ~45% del traffico — spinti dai prezzi bassi.

Volume di token ≠ fatturato: due metriche diverse.

Fonti: Menlo Ventures, "State of GenAI in the Enterprise" 2025 · OpenRouter rankings · rilevazione 22/06/2026. I numeri cambiano spesso.

La mappa della sfera genAI

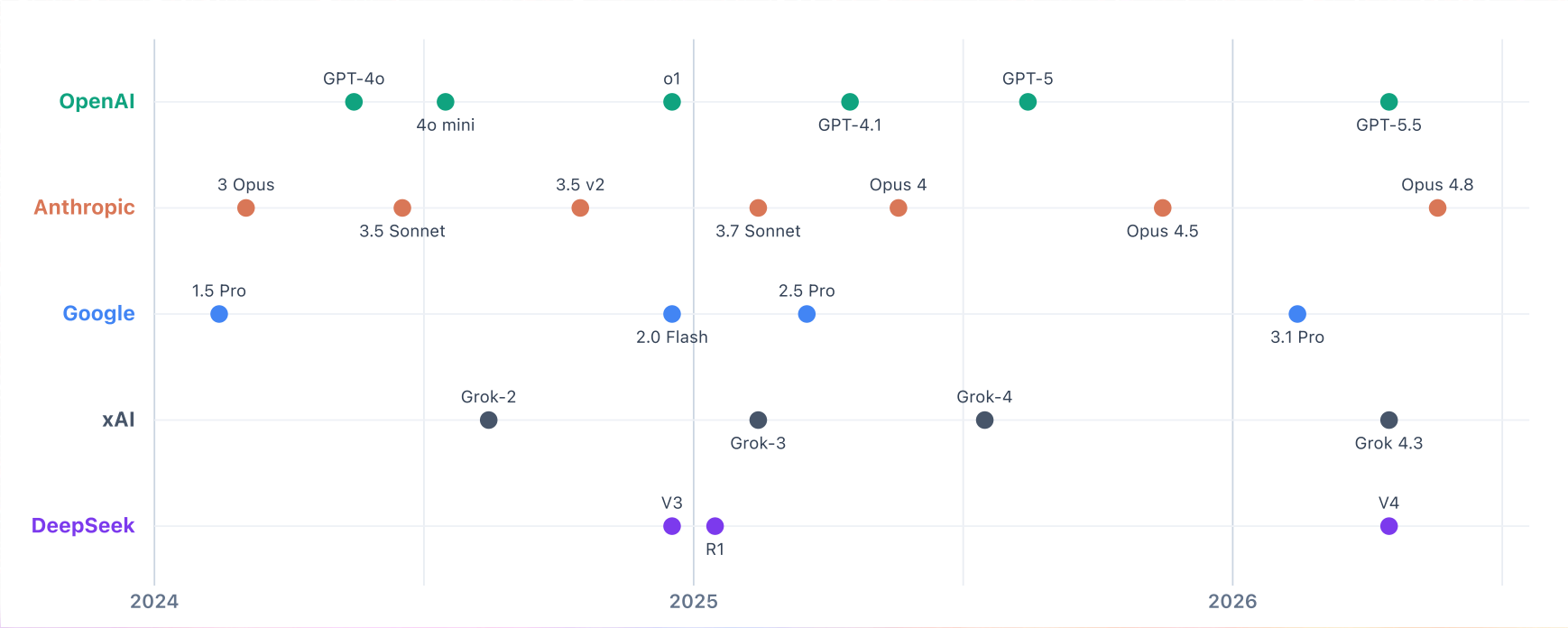
Chi copre quali aree. ● forte · ● parziale · ● assente

	Testo	Coding	Immagini	Video	Audio	Embed.	RAG
OpenAI	●	● Codex	●	●	●	●	● File Search
Anthropic	●	● Claude Code	●	●	●	●	●
Google	●	● Antigravity	●	● Veo	●	●	● File Search
xAI · Grok	●	● Grok Build	●	●	●	●	●

Video OpenAI: Sora ritirato (compute sul coding). Anthropic: embeddings via Voyage, nessun vector store gestito. Rilevazione 22/06/2026.

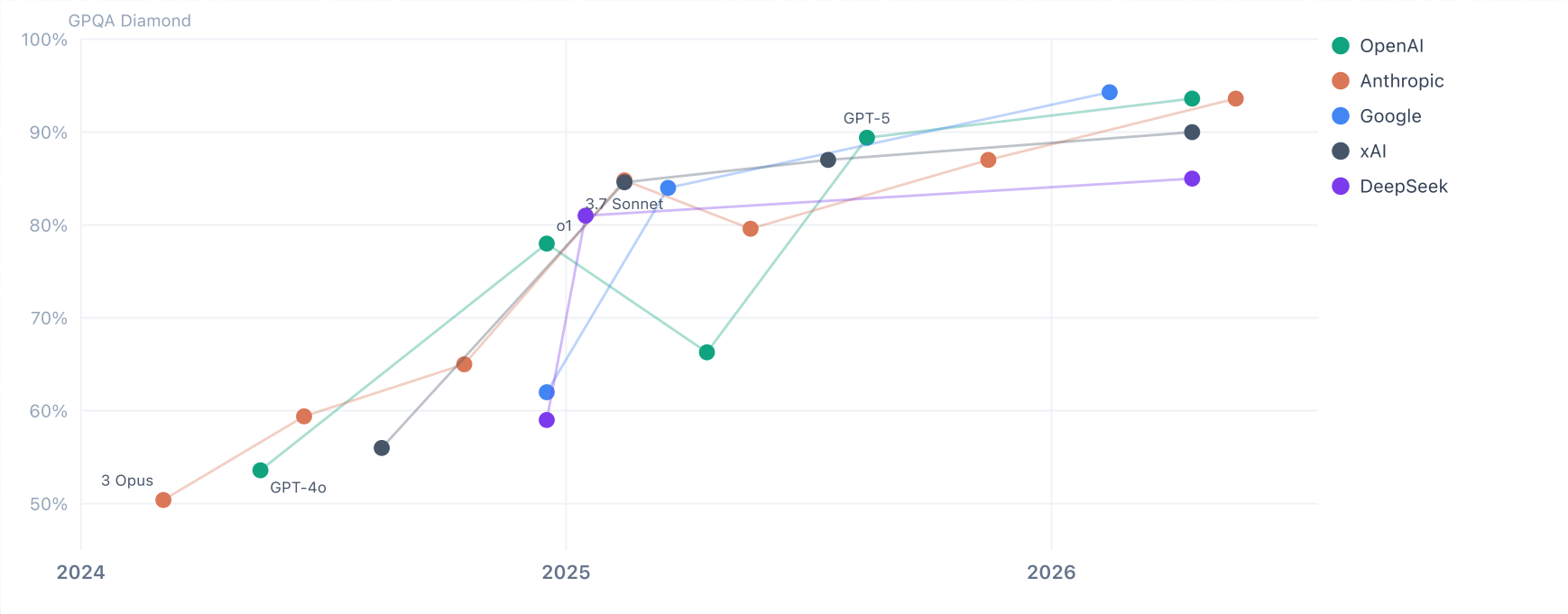
Quanto va veloce il campo

Le release dei modelli di punta dal 2024: una cadenza serrata, su tutti i provider.



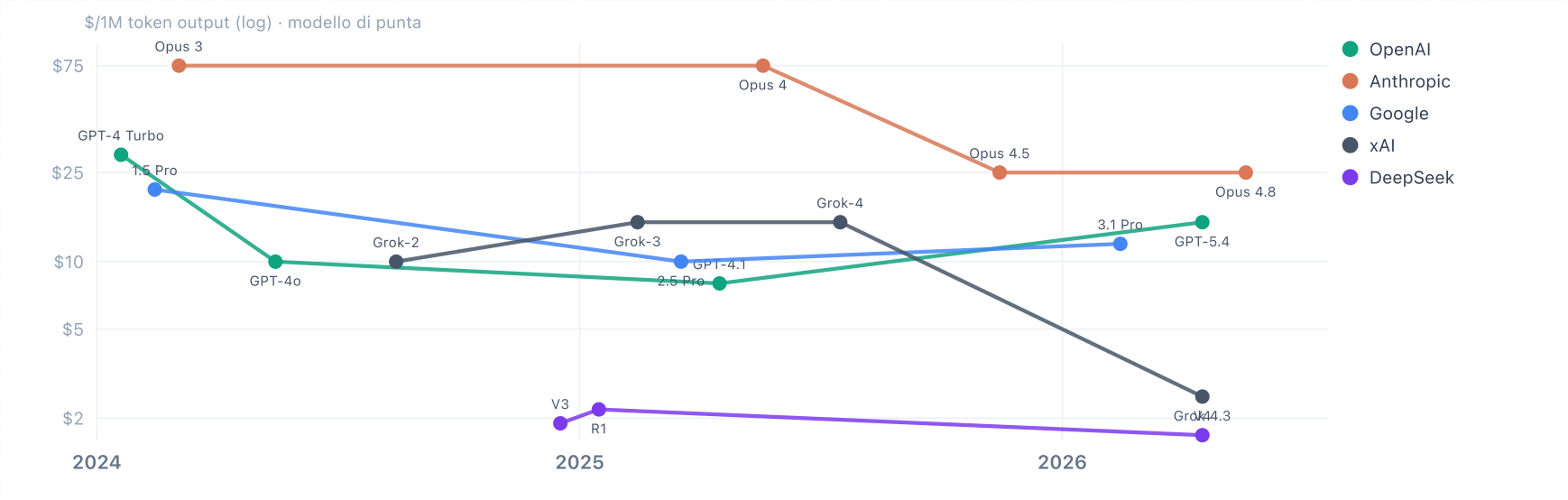
Sempre più bravi: le performance nel tempo

Quanto sono diventati bravi i modelli, nel tempo: la curva sale.



Sempre meno cari: i costi nel tempo

Una linea per provider, il modello di punta nel tempo: il prezzo scende, generazione dopo generazione.



CONCORRENZA
ogni nuovo modello economico abbassa il prezzo di riferimento.

EFFICIENZA
distillazione e MoE: modelli più piccoli, stessa capacità.

HARDWARE + INFERENZA
GPU migliori, quantizzazione, batching, caching.

MODELLI OPEN
i pesi liberi (DeepSeek, Llama) tengono basso il prezzo.

3. Quanto costa

Il modello pay-per-token



Paghi i token, in input e in output

Pay-per-use: niente canone, paghi in proporzione al testo che passa dal modello.

INPUT

Quello che mandi: istruzioni, domanda, contesto.



→ **MODELLO** →

OUTPUT

Quello che risponde. Spesso costa più dell'input.



I **prezzi di listino** li vediamo tra poco, in questa lezione (Cap 5). Come si **contano** davvero i token e come si **ottimizza** la spesa → prossima lezione, `L14 · Token & Costi`.

`IN BREVE` Il prezzo è proporzionale ai token che mandi e a quelli che ricevi.

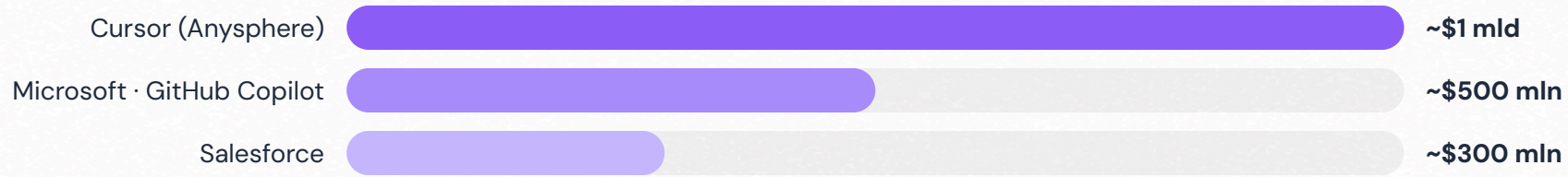
Quanto si spende in API

Per dare un'idea della scala. Cifre riportate/stimate dalla stampa: ordini di grandezza, non numeri ufficiali.

~$12,5 mld

spesi in API di foundation model nel 2025 (Menlo Ventures)

SPESA API ANNUALIZZATA · STIME PER AZIENDA



1.000+ aziende spendono **oltre $1M/anno** ciascuna (raddoppiate in due mesi).

Altri grandi utenti citati: Uber, Cognition (Devin), Netflix, Spotify…

Stime/run-rate da The Information, Fortune, Menlo Ventures, Sacra (2025–2026). Le aziende non dichiarano la spesa API: cifre approssimative, cambiano in fretta.

4. Quale modello per cosa

Punti di forza, per task



Forza per task: il quadro qualitativo

A grandi linee, chi è forte in cosa. È un quadro che cambia in fretta.

OpenAI

Ottimo per operazioni **generiche** di testo, multimodale e generazione **immagini**.

Anthropic

I migliori modelli per scrivere **codice** e per gli agenti (OpenAI subito dietro).

Google · Gemini

Comodo per tutte le operazioni **multimodali** e per l'ottima **integrazione nella suite Google**.

Nota per chi eroga: è un quadro qualitativo. La cosa importante non è "il migliore", ma scegliere il modello adatto al task (e al budget).

I numeri: le leaderboard

A supporto del quadro qualitativo, due classifiche citabili. Sono fotografie: cambiano in fretta.

LMArena · preferenza generale (Elo)

votazione umana testa a testa

Claude (Anthropic)	~1508
GPT-5.5 (OpenAI)	~1500
Gemini 3 (Google)	~1495

Cluster di testa stretto (~15-20 Elo).

SWE-bench Verified · coding

% di issue software risolte davvero

Claude Opus 4.8	~88%
GPT-5.5 / Codex	~80%+
Gemini 3 Pro	~76%

Anthropic in testa sul coding.

Fonti: LMArena (arena.ai) · SWE-bench Verified (leaderboard.steel.dev, agg. 28/05/2026) · rilevazione 22/06/2026.

5. I prezzi

Token, immagini, abbonamenti



Prezzi dei top model

USD per 1 milione di token, input / output. I tier: flagship, mid, small.

Provider	Flagship (in / out)	Mid (in / out)	Small (in / out)
OpenAI	`gpt-5.5` · $5 / $30	`gpt-5.4-mini` · $0.75 / $4.50	`gpt-5.4-nano` · $0.20 / $1.25
Anthropic	`claude-opus-4-8` · $5 / $25	`claude-sonnet-4-6` · $3 / $15	`claude-haiku-4-5` · $1 / $5
Google	`gemini-3.1-pro` · $2 / $12	`gemini-2.5-flash` · $0.30 / $2.50	`2.5-flash-lite` · $0.10 / $0.40
xAI	`grok-4.3` · $1.25 / $2.50	—	—

L'**output** costa quasi sempre più dell'input (es. Opus 4.8: $5 → $25).

Esistono tier ancora più potenti e cari (es. `gpt-5.5-pro` $30/$180).

Prezzi di listino standard · rilevazione 22/06/2026 (cambiano spesso).

Prezzi della generazione immagini

Solo chi ce l'ha. Anthropic non genera immagini.

Provider	Modello	Costo indicativo
OpenAI	`gpt-image-2`	a consumo, ~$0.01 – $0.17 per immagine (per qualità)
Google	`gemini-3-pro-image` ("Nano Banana")	~$0.13 per immagine
Google	`gemini-2.5-flash-image`	~$0.04 per immagine
Anthropic	—	nessuna generazione immagini

Le immagini si pagano a **token** come il testo: il costo dipende da risoluzione e qualità richieste.

Rilevazione 22/06/2026.

Abbonamenti dei coding agent

Eco della L12: l'abbonamento "a mano" è una cosa, l'API "a consumo" è un'altra — billing separati.

ChatGPT / Codex

Plus	$20/mese
Pro	$200/mese

Claude / Claude Code

Pro	$20/mese
Max 5×	$100/mese
Max 20×	$200/mese

Google · Gemini

AI Pro	$19.99/mese
AI Ultra	$99–199/mese

Abbonamento ≠ API. Il piano mensile è per usare l'app/agente a mano; l'API si paga a token. Rilevazione 22/06/2026.

6. La documentazione

Dove guardare, e da dove partiamo



Le doc delle API

Ogni provider ha la sua documentazione ufficiale. È sempre il primo posto dove guardare.

OpenAI
developers.openai.com/api/docs

Anthropic
docs.claude.com

Google · Gemini
ai.google.dev

xAI · Grok
docs.x.ai



Ci concentriamo su OpenAI

La parte tecnica — la chiave e il deep dive della response — la facciamo su OpenAI.

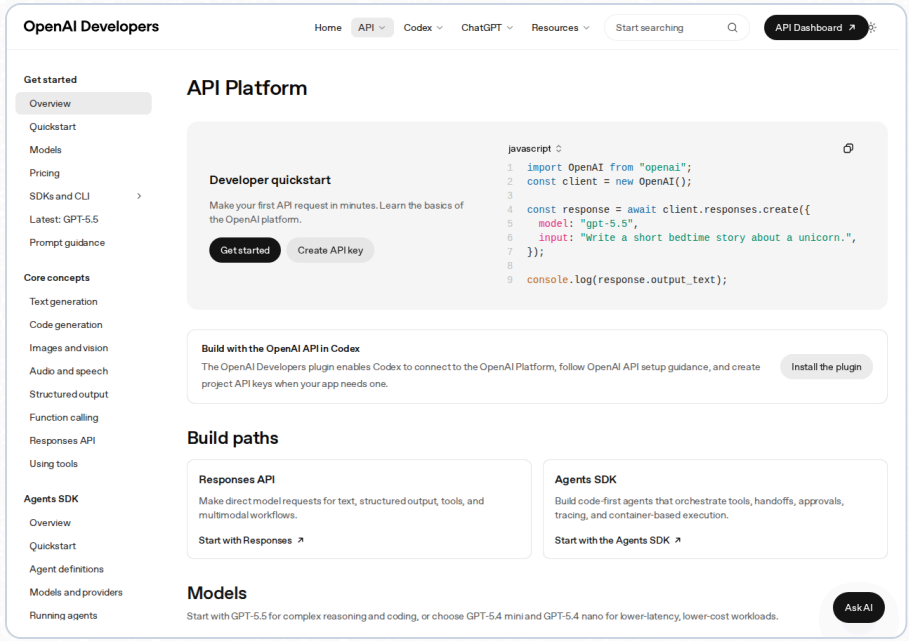
OpenAI · da qui in poi

`chiave` `endpoint` `request / response JSON` `parametri equivalenti`

Le altre API sono molto simili: stessa logica. Quello che vediamo su OpenAI si trasferisce quasi identico altrove (lo avevamo visto alla L12 con Anthropic).

Le docs di OpenAI: la mappa

Il primo posto dove guardare. Oggi vivono su developers.openai.com (platform.openai.com reindirizza qui).



- `Get started` overview, quickstart, modelli, prezzi, SDK
- `Guides` i concetti: testo, immagini, audio, structured output, function calling, **Responses API**, tools
- `Agents SDK` costruire agenti
- `Specialized models` immagini, video, embeddings, moderation
- `API reference` ogni endpoint, ogni parametro
- `Cookbook` ricette di codice pronte

Screenshot: developers.openai.com/api/docs/overview · rilevazione 22/06/2026.

Le guide che ci servono

Dentro "Guides": i mattoni del corso. Oggi ci concentriamo sulla Responses API.

TEXT GENERATION

generare e leggere testo: il punto di partenza.

RESPONSES API

l'interfaccia principale per le risposte del modello. Il cuore di oggi.

STRUCTURED OUTPUTS

forzare l'output in JSON conforme a uno schema.

FUNCTION CALLING · TOOLS

collegare il modello a funzioni e strumenti.

STREAMING

ricevere la risposta a pezzi, in tempo reale.

IMAGES · AUDIO · EMBEDDINGS

le altre modalità, una guida per ciascuna.



7. La chiave API

Crearla, e a cosa serve



Creare la chiave (richiamo L12)

Si crea dalla dashboard del provider. Su OpenAI: platform.openai.com.

01
Dashboard
accedi al provider

→

02
Crea API key
copiala subito (si vede una volta)

→

03
Nel .env
mai nel codice, mai su Git

```
OPENAI_API_KEY=sk-...      # nel file .env, che sta nel .gitignore
```

Conviene impostare un **tetto di spesa** sull'account, così un errore nel codice non svuota il credito.



A cosa serve la chiave

Tre funzioni, tutte insieme.

AUTH

Chi sei

Identifica chi sta chiamando l'API. Senza chiave valida, richiesta rifiutata (401).

BILLING

Su quale conto pagare

A quale account addebitare il consumo a token di quella richiesta.

RATE LIMITING

Quanto puoi spingere

Quante richieste e quanti token al minuto puoi fare. Oltre il limite: 429.

INSIGHT · USAGE TIER Più spendi, più puoi spendere: i provider ti promuovono di **tier** in base allo storico di spesa, e a ogni tier alzano i tuoi rate limit. All'inizio sei limitato; col tempo (e la spesa) i tetti salgono.

8. Dentro la response

Il JSON, campo per campo



La risposta è un JSON

Lo stesso oggetto che alla L12 avevamo guardato di sfuggita. Ora lo apriamo.

```
{
  "id": "resp_0acf ... ",
  "object": "response",
  "status": "completed",
  "model": "gpt-5.5-2026-04-24",
  "output": [
    { "type": "message", "role": "assistant",
      "content": [{ "type": "output_text",
        "text": "Ecco la risposta ... " }] }
  ],
  "usage": { "input_tokens": 19,
             "output_tokens": 31,
             "total_tokens": 50 },
  "temperature": 1.0,
  "max_output_tokens": null,
  "store": true,
  "text": { "verbosity": "medium" },
  "tools": []
}
```

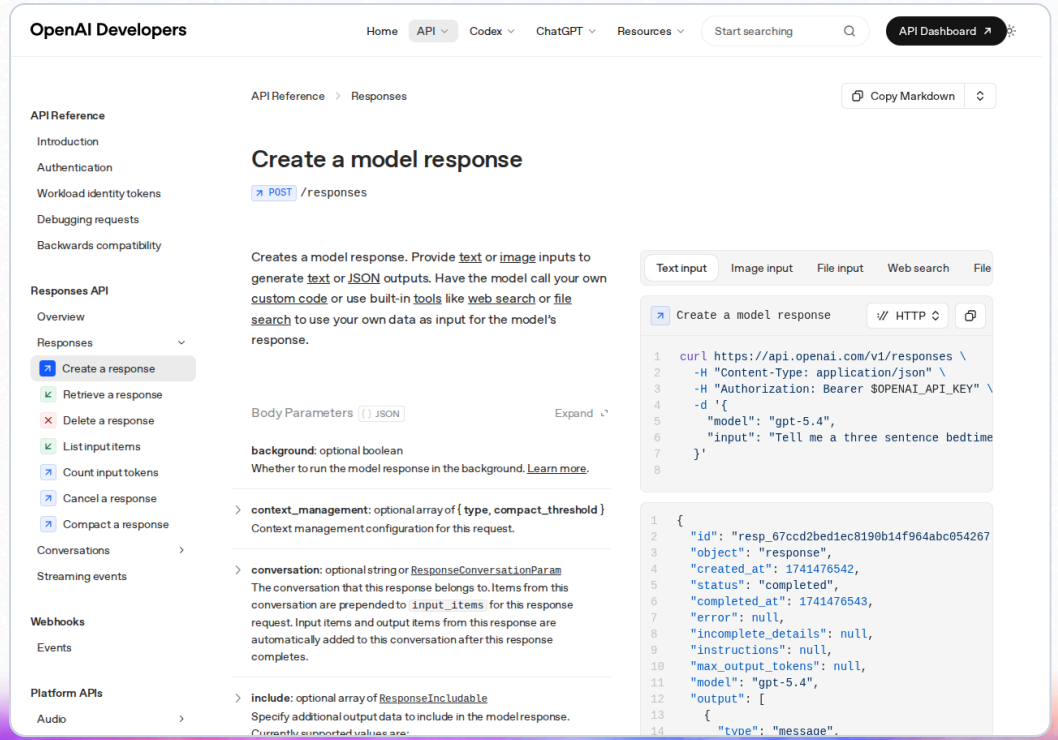
- `id / status` identificativo ed esito
- `model` il modello esatto usato
- `output` il testo generato
- `usage` i token usati = il costo
- `temperature, store, text…` i parametri usati

JSON abbreviato. Lo apriamo nelle prossime slide.



La Responses API nella reference

POST /v1/responses, ovvero Create a model response: apriamola e leggiamola parametro per parametro.



A sinistra i **parametri del body**: model, input, instructions…

A destra l'**esempio** di request e la response JSON.

La leggiamo insieme, parametro per parametro.

I parametri della richiesta

Tanti parametri; apriamo i principali, uno per uno. In viola quelli già usati alla L12.

`model`	`input`	`instructions`	`temperature`
`max_output_tokens`	`text.verbosity`	`text.format`	`reasoning.effort`
`tools`	`tool_choice`	`store`	`previous_response_id`
`stream`			

Ne esistono altri (`top_p` , `truncation` , `service_tier` , `metadata` , `include` …): qui i più importanti.



`model`

Quale modello chiamare. (già visto alla L12)

> **model**: optional `ResponsesModel`
>
> Model ID used to generate the response, like `gpt-4o` or `o3`. OpenAI offers a wide range of models with different capabilities, performance characteristics, and price points. Refer to the model guide to browse and compare available models.

```python
resp = client.responses.create(
    model="gpt-5.4",      # l'ID del modello
    input="Ciao!",
)
```

OUTPUT

`resp.model` → `"gpt-5.4-2026-...."` : la versione esatta usata.

Stringa con l'ID del modello. Conviene pinnare la versione datata per avere risultati riproducibili.

`developers.openai.com/api/reference/responses · rilevazione 22/06/2026`



`input`

Cosa chiediamo: testo o lista di messaggi. (già visto)

> **input:** optional string or array of `EasyInputMessage` or `object` or `ResponseOutputMess…`
>
> Text, image, or file inputs to the model, used to generate a response.
>
> Learn more:
>
> - Text inputs and outputs
> - Image inputs
> - File inputs
> - Conversation state
> - Function calling

```python
# stringa semplice
client.responses.create(model="gpt-5.4", input="Riassumi: ...")

# oppure lista di messaggi/contenuti
client.responses.create(model="gpt-5.4", input=[
    {"role": "user", "content": "Ciao"},
])
```

NOTA

Può contenere anche immagini e file, non solo testo (multimodale).

Stringa per il caso semplice; array di messaggi quando vuoi ruoli, storia o contenuti misti.



`instructions`

Le istruzioni di sistema: il system prompt. (già visto)

instructions: optional string

A system (or developer) message inserted into the model's context.

When using along with `previous_response_id`, the instructions from a previous response will not be carried over to the next response. This makes it simple to swap out system (or developer) messages in new responses.

```
client.responses.create(
    model="gpt-5.4",
    instructions="Rispondi sempre in italiano, conciso.",
    input="What is RAG?",
)
```

OUTPUT

Risposta in italiano e sintetica, qualunque sia la lingua dell'input.

Inserite in cima al contesto del modello: definiscono tono, ruolo e regole della risposta.

`developers.openai.com/api/reference/responses · rilevazione 22/06/2026`



`temperature`

Quanto varia l'output. (già visto alla L12)

temperature: optional number
What sampling temperature to use, between 0 and 2. Higher values like 0.8 will make the output more random, while lower values like 0.2 will make it more focused and deterministic. We generally recommend altering this or `top_p` but not both.
minimum: `0`, maximum: `2`

```
client.responses.create(
    model="gpt-5.4",
    input="Uno slogan per un caffè",
    temperature=0.2,    # 0 = stabile · 2 = creativo
)
```

OUTPUT
`0.2` → output quasi ripetibile · `1.5` → più vario e creativo.

Range 0–2. Regola questo **oppure** top_p, non entrambi.

`developers.openai.com/api/reference/responses · rilevazione 22/06/2026`



`max_output_tokens`

Tetto massimo ai token generati in risposta.

max_output_tokens: optional number
An upper bound for the number of tokens that can be generated for a response, including visible output tokens and reasoning tokens.
minimum: `16`

```
client.responses.create(
    model="gpt-5.4",
    input="Spiega i transformer",
    max_output_tokens=200,
)
```

OUTPUT

Se la risposta viene tagliata: `status: "incomplete"` (reason: max_output_tokens).

Include anche i token di reasoning. Minimo 16. Utile per tenere sotto controllo costo e lunghezza.

`developers.openai.com/api/reference/responses · rilevazione 22/06/2026`



`text.verbosity`

Quanto deve essere prolissa la risposta finale.

> **text:** optional `ResponseTextConfig` { **format, verbosity** }
>
> Configuration options for a text response from the model. Can be plain text or structured JSON data. Learn more:
>
> - Text inputs and outputs
> - Structured Outputs

OUTPUT

`low` → risposta secca · `high` → spiegazione estesa.

Vive dentro l'oggetto `text`, che configura il formato dell'output testuale.

```
client.responses.create(
    model="gpt-5.4",
    input="Cos'è un'API?",
    text={"verbosity": "low"},   # low | medium | high
)
```

`developers.openai.com/api/reference/responses · rilevazione 22/06/2026`



text.format · Structured Outputs

Forzare l'output a un JSON conforme a uno schema.

```
client.responses.create(
    model="gpt-5.4",
    input="Estrai nome ed eta: Mario, 30 anni",
    text={"format": {"type": "json_schema", "name": "persona",
        "schema": {"type": "object", "properties": {
            "nome": {"type": "string"},
            "eta":  {"type": "integer"}},
        "required": ["nome", "eta"]}}},
)
```

OUTPUT

`{"nome": "Mario", "eta": 30}` — JSON **garantito** conforme allo schema.

Niente più parsing fragile di testo libero: l'output è dati strutturati, pronti da usare.

developers.openai.com/api/reference/responses · rilevazione 22/06/2026



reasoning.effort

Quanto far 'ragionare' i modelli reasoning prima di rispondere.

> **reasoning:** optional Reasoning { **context, effort, generate_summary, summary** }
>
> **gpt-5 and o-series models only**
>
> Configuration options for reasoning models.

```
client.responses.create(
    model="gpt-5.4",
    input="Risolvi questo problema passo passo: ...",
    reasoning={"effort": "high"},   # minimal → high
)
```

OUTPUT

Effort alto → più token di reasoning: più qualità sui task difficili, ma più lento e caro.

Solo per i modelli che ragionano. È il bilanciere tra qualità e costo/latenza.

developers.openai.com/api/reference/responses · rilevazione 22/06/2026



tools + tool_choice

Strumenti attivabili e come decidere se usarli.

> **tools:** optional array of `object` or `object` or `object` or 12 more
>
> An array of tools the model may call while generating a response. You can specify which tool to use by setting the `tool_choice` parameter.
>
> We support the following categories of tools:
>
> - **Built-in tools:** Tools that are provided by OpenAI that extend the model's capabilities, like web search or file search. Learn more about built-in tools.
> - **MCP Tools:** Integrations with third-party systems via custom MCP servers or

```
client.responses.create(
    model="gpt-5.4",
    input="Che tempo fa a Milano?",
    tools=[{"type": "web_search"}],
    tool_choice="auto",   # none | auto | required
)
```

OUTPUT

Il modello chiama `web_search`, legge il risultato e risponde aggiornato.

`tools`: built-in (web/file search, MCP) o funzioni tue.
`tool_choice`: se e quali usare.

`developers.openai.com/api/reference/responses · rilevazione 22/06/2026`



`store + previous_response_id`

Salvare la response lato server e concatenare i turni.

previous_response_id: optional string
The unique ID of the previous response to the model. Use this to create multi-turn conversations. Learn more about conversation state. Cannot be used in conjunction with `conversation`.

```python
r1 = client.responses.create(model="gpt-5.4",
    input="Mi chiamo Anna", store=True)

r2 = client.responses.create(model="gpt-5.4",
    input="Come mi chiamo?",
    previous_response_id=r1.id)
```

OUTPUT

`r2` 'ricorda' il turno precedente senza re-inviare tutta la storia.

`store` conserva la response sul server; `previous_response_id` aggancia il turno prima.

`developers.openai.com/api/reference/responses · rilevazione 22/06/2026`



`stream`

Ricevere la risposta a pezzi, in tempo reale.

stream: optional boolean
If set to true, the model response data will be streamed to the client as it is generated using server-sent events. See the Streaming section below for more information.

```python
stream = client.responses.create(
    model="gpt-5.4", input="Scrivi una poesia", stream=True)

for event in stream:
    if event.type == "response.output_text.delta":
        print(event.delta, end="")
```

OUTPUT

Il testo arriva token per token (server-sent events): UX da chat, niente attesa del blocco intero.

Utile per interfacce in tempo reale. Cambia come leggi la risposta: eventi, non un singolo oggetto.

developers.openai.com/api/reference/responses · rilevazione 22/06/2026



L’oggetto response (1/2)

Cosa torna indietro. La struttura che alla L12 avevamo solo sfiorato.

`id` string
identificativo della response (per concatenarla).

`status` string
`completed` , `incomplete` , `failed` …

`model` string
il modello esatto (versione datata).

`output[]` array
i contenuti generati: dentro, `content → output_text` .

`output_text` scorciatoia SDK
il testo già estratto da `output[]` — è quello che stampavamo alla L12 con `response.output_text` .

`developers.openai.com/api/reference/responses · Response object · rilevazione 22/06/2026`



usage: i token che paghi (2/2)

Il campo che fa da ponte verso i prezzi e verso la prossima lezione.

```
"usage": {
  "input_tokens": 19,
  "output_tokens": 31,
  "total_tokens": 50
}
```

`input_tokens` quanti token sono entrati

`output_tokens` quanti ne ha generati

`total_tokens` la somma: **è ciò che paghi**

→ moltiplicato per il **prezzo** del modello (Cap 5)

→ come si **contano** e si ottimizzano: L14

`IN BREVE` usage è il contatore dei token: da lì nasce il conto.



Riepilogo

Quello che ci portiamo a casa stasera.

- Training (i big) vs inferenza (noi)
- Closed-source vs open-weights
- GPU e perché conviene l'API
- Mercato e aree di forza
- Pay-per-token
- Quale modello per cosa
- Tabelle prezzi
- La chiave: auth / billing / rate limit
- **Dentro la response: output + usage**



Prossime lezioni

Il Modulo 2 continua da qui.

PROSSIMA · L14

Token & Costi

Come si **contano** davvero i token (tiktoken), come si calcola e si **ottimizza** la spesa.

PIÙ AVANTI

AI Agent loop

Il ciclo di un agente: il modello chiama strumenti, osserva i risultati e itera fino alla soluzione.

Il `usage` di stasera è il filo diretto verso la L14.

Kahoot

Cinque domande sulla lezione di stasera.

STUDENTI

Entrate su `kahoot.it`

e scrivete il **PIN** che compare a schermo.

DOCENTE · AVVIA LA PARTITA

`play.kahoot.it/v2/lobby?quizId=…`

link host inserito a fine lavorazione

Argomenti: training vs inferenza · closed vs open weights · pay-per-token · a cosa serve la chiave · dentro la response.